

URL Shortener (TinyURL)

High-Level Design study note · interview answer + alternatives & production reality

1 · Requirements

Functional

- Given a long URL, return a short URL (unique short code).
- Given a short URL, redirect to the original long URL.
- Optional: custom alias, link expiry, click analytics, user accounts.

Non-functional

- High availability — a broken redirect is a broken product.
- Low latency on redirects (the hot path); reads dominate.
- Short codes must be globally unique and ideally hard to guess.
- Horizontally scalable; heavily read-skewed (a few links go viral).

Read:write ratio is roughly **100:1** — the whole design optimises the read/redirect path.

2 · Back-of-envelope estimate

State your assumptions, then derive. Example numbers:

Quantity	Assumption / derivation	Result
New URLs (writes)	100M / month = 100M / 2.6M sec	~ 40 writes/sec
Redirects (reads)	100:1 read:write ratio	~ 4,000 reads/sec
Storage (5 yr)	100M/mo x 60 mo x ~500 bytes/record	~ 3 TB
Keyspace (7 char)	62^7 combinations	~ 3.5 trillion

These numbers justify later choices: modest write rate (a single counter/allocator is fine), high read rate + skew (cache absorbs most reads), and a keyspace of 3.5T that comfortably outlasts the 5-year record count.

3 · API

Endpoint	Purpose	Returns
POST /shorten { longUrl, alias? }	Create a short code	201 { shortUrl }
GET /{code}	Resolve & redirect	301 / 302 -> Location: longUrl
GET /{code}/stats	Analytics (optional)	200 { clicks, ... }

4 · Data model

Field	Type	Note
id / code	PK	the short code (or its numeric id) — the entire read path keys on this
longUrl	string	destination
createdAt	timestamp	for expiry / auditing

Field	Type	Note
expiresAt	timestamp?	optional TTL
userId	string?	owner, for analytics / quotas

Access pattern is a single point-lookup by code — no joins, no range scans. That shape drives the datastore choice below.

5 · Core problem — generating the short code

This is the heart of the design. The requirement splits into two *separate* properties that people often conflate: **uniqueness without a lookup**, and **short + unguessable**. No single tool nails both for free — know the trade of each.

Option A — Snowflake ID, base62-encoded

A 64-bit ID assembled per-server, then base62-encoded (~11 chars).

```
[ 41 bits timestamp ][ 10 bits machine id ][ 12 bits sequence ]
```

- **Pros:** zero coordination per request, very high throughput, roughly time-ordered.
- **Cons:** ~11 chars (not 7); timestamp sits high, so codes are near-sequential and **guessable**; must handle **clock rollback** (reject/wait if clock goes backwards) and **unique machine-id assignment** (10 bits = 1024 ids, via config or ZooKeeper/etcd).
- **Best for:** internal object ids (Twitter/Discord style), where length & guessability don't matter.

Option B — Counter + base62 (range allocation)

- A monotonic counter, base62-encoded; each server is handed a **range/block** (e.g. 1–1,000,000) by a small allocator (ZooKeeper/etcd or a ticket table), then burns through it locally.
- **Pros:** genuinely short (7 chars until 3.5T), no per-request coordination, conceptually simple.
- **Cons:** still sequential / guessable; needs a range allocator; a single un-ranged counter would be a write bottleneck.

Option C — Key Generation Service (KGS)

- Pre-generate **random** unique 7-char keys offline into an *unused-keys* table; app servers pull keys in batches and mark them used.
- **Pros:** short **and** unguessable, key length fully decoupled from any id size, no lookup on the write path.
- **Cons:** one more stateful service; must atomically hand out keys (avoid double-issue) and monitor pool exhaustion.

Option D — Hash of the long URL

- Take MD5/SHA-256 of the long URL, keep the first 6–8 base62 chars.
- **Pros:** identical URLs collapse to the same code (natural dedupe).
- **Cons:** truncation causes **collisions** — must detect and re-salt/retry; not ordered.

Interview pick If the rubric wants short + unguessable, lead with **KGS**; if they want the simplest defensible scheme, **counter + base62 with range allocation**. Mention Snowflake as the zero-coordination option and note its length/guessability cost. Naming all four and stating the trade is what scores.

6 · Redirect — 301 vs 302

	301 Permanent	302 Temporary
Browser caches redirect	Yes — fewer hits reach you	No — every hit comes to you
Server load	Lower	Higher
Click analytics	Lost (cached hits invisible)	Captured (every hit seen)

Interview pick 302 in almost all real shorteners — the click data *is* the business (bit.ly, marketing, QR). Choose 301 only for a pure redirect where analytics don't matter and you want the caching win.

7 · Load balancing

App servers are **stateless** (state lives in shared Redis + DB), so any server handles any request. Use a standard L7 balancer (ALB / Nginx / Envoy) with **round-robin** or **least-connections**.

Common trap: don't put **consistent hashing at the load balancer** for stateless servers — it pins requests for no gain. Consistent hashing belongs at the **data tier** (sharding cache/DB), where nodes own a slice of data and you want minimal key movement when nodes change. (Exception: if each app server held its own *local* cache, LB-level hashing would raise local hit rates — not the case with shared Redis.)

8 · Datastore

Access is a single-key lookup, read-heavy — a perfect KV shape.

- **Interview answer:** NoSQL KV store — DynamoDB or Cassandra — with code as the partition key. Scales horizontally, no joins needed.
- **Production reality:** a large share of real shorteners run on **PostgreSQL / MySQL**. The dataset is smaller than people assume and an indexed primary-key lookup is trivially fast. Teams move to Cassandra/Dynamo only once write throughput or dataset size genuinely outgrows a single relational primary.

Interview pick Say both: “NoSQL KV is the scalable choice; in practice I'd start relational and migrate when I've *proven* I need to.” That phrasing reads as experienced rather than memorised.

9 · Caching

- **Redis** in front of the DB, mapping code -> longUrl. Universal choice, no asterisk.
- Strategy: **cache-aside** with a TTL; **LRU** eviction. Traffic skew (viral links) means a modest cache absorbs the large majority of reads.
- For the very hottest links, a **CDN / edge** can serve the redirect even closer to the user.

10 · Request paths

Write client -> LB -> app server -> get code (KGS / counter / Snowflake) -> store { code, longUrl, meta } in DB -> return short URL.

Read client GET /{code} -> LB -> app server -> check Redis (hit: redirect) (miss: DB lookup -> populate cache -> redirect) -> 301/302 to longUrl; 404 if missing/expired. Emit an analytics event **asynchronously** so it never blocks the redirect.

11 · Scaling & sharding

- Shard DB and cache by hash (code) using **consistent hashing** (with virtual nodes) so few keys move when nodes are added/removed.
- Read replicas for extra read capacity; primary handles the low write rate.
- Multi-region via geo-DNS for global low latency.
- Analytics off the hot path: push click events to **Kafka** -> a warehouse, rather than incrementing a counter inline.

12 · Other concerns to have ready

- **Rate limiting** on create (per user / IP) to stop abuse.
- **Custom aliases:** check uniqueness and reserve atomically; reject clashes.
- **Expiry & cleanup:** TTL field + background sweep (or DB-native TTL).
- **Security:** random keys to avoid guessing; scan destination URLs for malware/phishing.
- **Dedupe (optional):** a hash lookup can return the existing code for an already-shortened URL.

13 · Decision summary — interview pick vs alternatives

Decision	Interview pick	Alternatives / in use
Code generation	KGS (short + unguessable), or counter+base62	Snowflake (zero-coord, ~11ch, guessable); hash-of-URL (dedupe, collisions)
Code length / encoding	7 chars base62 (~3.5T space)	~11 chars if Snowflake; url-safe base64
Redirect	302 (keep analytics)	301 (cacheable, fewer hits, loses analytics)
Load balancer	Round-robin / least-connections (L7)	Consistent hashing only if servers hold local state
Datastore	NoSQL KV (DynamoDB / Cassandra)	PostgreSQL / MySQL — common in practice at small/mid scale
Cache	Redis, cache-aside + TTL	Memcached; CDN edge for hottest links
Uniqueness coordination	Pre-allocated ranges / KGS pool	ZooKeeper / etcd for ranges & machine-id assignment
Analytics	Async event stream (Kafka) -> warehouse	Inline counter increment (simpler, adds write latency)

Rule of thumb — interview HLD optimises for showing you know the *scalable* tool; production optimises for the *simplest* thing that meets the requirement, then scales when forced. Being able to state both, and why one concern (guessability, ops simplicity, analytics) outranks the textbook optimisation, is what distinguishes an experienced answer.